

HBase

Apache HBase is made for big jobs, like a nail gun. You would never use HBase to catalog your corporate sales list or build a to-do list app for fun, just like you'd never use a nail gun to build a doll house. If the size of your dataset isn't many, many gigabytes at the very least then you should probably use a less heavy-duty tool.

At first glance, HBase looks a lot like a relational database, so much so that if you didn't know any better, you might think that it is one. In fact, the most challenging part of learning HBase isn't the technology; it's that many of the words used in HBase are deceptively familiar. For example, HBase stores data in buckets it calls *tables*, which contain *cells* that appear at the intersection of *rows* and *columns*. Sounds like a relational database, right?

Wrong! In HBase, tables don't behave like relations, rows don't act like records, and columns are completely variable and not enforced by any predefined schema. Schema design is still important, of course, because it informs the performance characteristics of the system, but it won't keep your house in order—that task falls to you and how your applications use HBase. In general, trying to shoehorn HBase into an RDBMS-style system is fraught with nothing but peril and a certain path to frustration and failure. HBase is the evil twin, the bizarro doppelgänger, if you will, of RDBMS.

On top of that, unlike relational databases, which sometimes have trouble scaling out, HBase doesn't scale *down*. If your production HBase cluster has fewer than five nodes, then, quite frankly, you're doing it wrong. HBase is not the right database for some problems, particularly those where the amount of data is measured in megabytes, or even in the low gigabytes.

So why would you use HBase? Aside from scalability, there are a few reasons. To begin with, HBase has some built-in features that other databases lack,

such as versioning, compression, garbage collection (for expired data), and in-memory tables. Having these features available right out of the box means less code that you have to write when your requirements demand them. HBase also makes strong consistency guarantees, making it easier to transition from relational databases for some use cases. Finally, HBase guarantees atomicity at the row level, which means that you can have strong consistency guarantees at a crucial level of HBase's data model.

For all of these reasons, HBase really shines as the cornerstone of a large-scale online analytics processing system. While individual operations may sometimes be slower than equivalent operations in other databases, scanning through enormous datasets is an area where HBase truly excels. For genuinely big queries, HBase often outpaces other databases, which helps to explain why HBase is often used at big companies to back heavy-duty logging and search systems.

Introducing HBase

HBase is a *column-oriented* database that prides itself on its ability to provide both consistency and scalability. It is based on Bigtable, a high-performance, proprietary database developed by Google and described in the 2006 white paper “Bigtable: A Distributed Storage System for Structured Data.”¹ Initially created for natural language processing, HBase started life as a contrib package for Apache Hadoop. Since then, it has become a top-level Apache project.



Luc says:

Hosted HBase with Google Cloud Bigtable

As you'll see later in this chapter, HBase can be tricky to administer. Fortunately, there's now a compelling option for those who want to utilize the power of HBase with very little operational burden: Google's Cloud Bigtable, which is part of its Cloud Platform suite of products. Cloud Bigtable isn't 100% compatible with HBase but as of early 2018 it's very close—close enough that you may be able to migrate many existing HBase applications over.

If you find the basic value proposition of, for example, Amazon's cloud-based DynamoDB compelling and you think HBase is a good fit for a project, then Cloud Bigtable might be worth checking out. You can at least be assured that it's run by the same company that crafted the concepts behind HBase (and the folks at Google do seem to know a thing or two about scale).

1. <http://research.google.com/archive/bigtable.html>

On the architecture front, HBase is designed to be fault tolerant. Hardware failures may be uncommon in individual machines but, in large clusters, node failure is the norm (as are network issues). HBase can gracefully recover from individual server failures because it uses both *write-ahead logging*, which writes data to an in-memory log before it's written (so that nodes can use the log for recovery rather than disk), and *distributed configuration*, which means that nodes can rely on each other for configuration rather than on a centralized source.

Additionally, HBase lives in the Hadoop ecosystem, where it benefits from its proximity to other related tools. Hadoop is a sturdy, scalable computing platform that provides a distributed file system and MapReduce capabilities. Wherever you find HBase, you'll find Hadoop and other infrastructural components that you can use in your own applications, such as Apache Hive, a data warehousing tool, and Apache Pig, a parallel processing tool (and *many* others).

Finally, HBase is actively used and developed by a number of high-profile companies for their "Big Data" problems. Notably, Facebook uses HBase for a variety of purposes, including for Messages, search indexing, and stream analysis. Twitter uses it to power its people search capability, for monitoring and performance data, and more. Airbnb uses it as part of their realtime stream processing stack. Apple uses it for...something, though they won't publicly say what. The parade of companies using HBase also includes the likes of eBay, Meetup, Ning, Yahoo!, and many others.

With all of this activity, new versions of HBase are coming out at a fairly rapid clip. At the time of this writing, the current stable version is 1.2.1, so that's what you'll be using. Go ahead and download HBase, and we'll get started.

Day 1: CRUD and Table Administration

Today's goal is to learn the nuts and bolts of working with HBase. You'll get a local instance of HBase running in standalone mode (rather than in distributed mode), and then you'll use the HBase shell to create and alter tables and to insert and modify data using basic CRUD-style commands. After that, you'll explore how to perform some of those operations programmatically by using the HBase Java API in JRuby. Along the way, you'll uncover some HBase architectural concepts, such as the relationship between rows, column families, columns, and values in a table. Just bear in mind that these concepts in HBase are subtly different from their counterparts in relational databases.

According to most HBase admins out there, a fully operational, production-quality HBase cluster should really consist of no fewer than five nodes. But

this bulky of a setup would be overkill for our needs (and for our laptops). Fortunately, HBase supports three running modes:

- *Standalone* mode is a single machine acting alone.
- *Pseudo-distributed* mode is a single node pretending to be a cluster.
- *Fully distributed* mode is a cluster of nodes working together.

For most of this chapter, you'll be running HBase in standalone mode. Yet even that can be a bit of a challenge, especially compared to other databases in this book, such as Redis or MongoDB. So although we won't cover every aspect of installation and administration, we'll give some relevant troubleshooting tips where appropriate.

Configuring HBase

Before you can use HBase, you need to provide it with some configuration, as HBase doesn't really have an “out-of-the-box” mode. Configuration settings for HBase are kept in a file called `hbase-site.xml`, which can be found in the `${HBASE_HOME}/conf` directory. Note that `HBASE_HOME` is an environment variable pointing to the directory where you've installed HBase. Make sure to set this variable now, preferably in your `.bash_profile` or similar file so that it persists across shell sessions.

Initially, this `hbase-site.xml` file contains just an empty `<configuration>` tag. You can add any number of property definitions to your configuration using this format:

```
<property>
  <name>some.property.name</name>
  <value>A property value</value>
</property>
```

The `hbase-default.xml` File

Another way of installing HBase is to clone the project directory locally using Git, either from the Apache repository at <http://git.apache.org/hbase.git> or from the mirror repository on GitHub at <https://github.com/apache/hbase>.

If you install HBase this way, you'll find an `hbase-default.xml` file in the `hbase-common/src/main/resources` subdirectory. This is a very handy file that lists all of the available configurable parameters for HBase—and there are many!—along with default values and descriptions for each parameter.

You can also see the contents of this file in your browser on GitHub.^a

a. <https://github.com/apache/hbase/blob/master/hbase-common/src/main/resources/hbase-default.xml>

By default, HBase uses a temporary directory to store its data files. This means you'll lose *all your data* whenever the operating system decides to reclaim the disk space. To keep your data around, you should specify a non-ephemeral storage location. Set the `hbase.rootdir` property to an appropriate path like so:

```
<property>
  <name>hbase.rootdir</name>
  <value>file:///path/to/hbase</value>
</property>
```

Here's an example configuration:

```
<property>
  <name>hbase.rootdir</name>
  <value>file://</value>
</property>
```

To start HBase, open a terminal (command prompt) and run this command:

```
$ ${HBASE_HOME}/bin/start-hbase.sh
```

To shut down HBase at any time, use the `stop-hbase.sh` command in the same directory.

If anything goes wrong, take a look at the most recently modified files in the `${HBASE_HOME}/logs` directory. On *nix-based systems, the following command will pipe the latest log data to the console as it's written:

```
$ cd ${HBASE_HOME}
$ find ./logs -name "hbase-*.log" -exec tail -f {} \;
```

The HBase Shell

The HBase shell is a JRuby-based command-line program you can use to interact with HBase. In the shell, you can add and remove tables, alter table schemas, add or delete data, and perform a bunch of other tasks. Later, we'll explore other means of connecting to HBase, but for now the shell will be our home.

With HBase running, open a terminal and fire up the HBase shell:

```
$ ${HBASE_HOME}/bin/hbase shell
```

To confirm that it's working properly, try asking it for version information. That should output a version number and hash, and a timestamp for when the version was released.

```
hbase> version
1.2.1, r8d8a7107dc4ccbf36a92f64675dc60392f85c015, Wed Mar 30 11:19:21 CDT 2016
```

You can enter help at any time to see a list of available commands or to get usage information about a particular command.

Next, execute the status command to see how your HBase server is holding up.

```
hbase> status
```

```
1 active master, 0 backup masters, 1 servers, 0 dead, 2.0000 average load
```

If an error occurs for any of these commands or if the shell hangs, a connection problem could be to blame. HBase does its best to automatically configure its services based on your network setup, but sometimes it gets it wrong. If you're seeing these symptoms, check the HBase network settings.

HBase Network Settings

By default, HBase tries to make its services available to external clients, but in our case, we only need to connect from the same machine. So it might help to add some or all of the following properties to your `hbase-site.xml` file (your mileage may vary). Note that the values in the following table will help only if you plan to connect locally and not remotely:

property name	value
<code>hbase.master.dns.interface</code>	<code>lo</code>
<code>hbase.master.info.bindAddress</code>	<code>127.0.0.1</code>
<code>hbase.regionserver.info.bindAddress</code>	<code>127.0.0.1</code>
<code>hbase.regionserver.dns.interface</code>	<code>lo</code>
<code>hbase.zookeeper.dns.interface</code>	<code>lo</code>

The properties tell HBase how to establish connections for the master server and region servers (both of which we'll discuss later) and for ZooKeeper (which HBase uses as a configuration service). The properties with the value "lo" refer to the so-called *loopback* interface. On *nix systems, the loopback interface is not a real network interface (like your Ethernet or wireless cards) but rather a software-only interface for the computer to use to connect to itself. The `bindAddress` properties tell HBase which IP address to try to listen on.

Creating a Table

Most programming languages have some concept of a key/value *map*. JavaScript has objects, Ruby has hashes, Go has maps, Python has dictionaries, Java has hashmaps, and so on. A table in HBase is basically a big map—well, more accurately, a *map of maps*.

In an HBase table, keys are arbitrary strings that each map to a *row* of data. A row is itself a map in which keys are called *columns* and values are stored

as uninterpreted arrays of bytes. Columns are grouped into *column families*, so a column's full name consists of two parts: the column family name and the *column qualifier*. Often these are concatenated together using a colon (for example, family:qualifier).

Here's what a simple HBase table might look like if it were a Python dictionary:

```
hbase_table = {
    # Table
    'row1': {
        # Row key
        'cf1:col1': 'value1', # Column family, column, and value
        'cf1:col2': 'value2',
        'cf2:col1': 'value3'
    },
    'row2': {
        # More row data
    }
}

queried_value = hbase_table['row1']['cf1:col1'] # 'value1'
```

For a more visual illustration, take a look at the following diagram.

	row keys	column family "color"	column family "shape"
row	"first"	"red": "#F00" "blue": "#00F" "yellow": "#FF0"	"square": "4"
row	"second"		"triangle": "3" "square": "4"

In this figure, we have a hypothetical table with two column families: color and shape. The table has two rows—denoted by dashed boxes—identified by their row keys: first and second. Looking at just the first row, you see that it has three columns in the color column family (with qualifiers red, blue, and yellow) and one column in the shape column family (square). The combination of row key and column name (including both family and qualifier) creates an address for locating data. In this example, the tuple first/color:red points us to the value '#F00'.

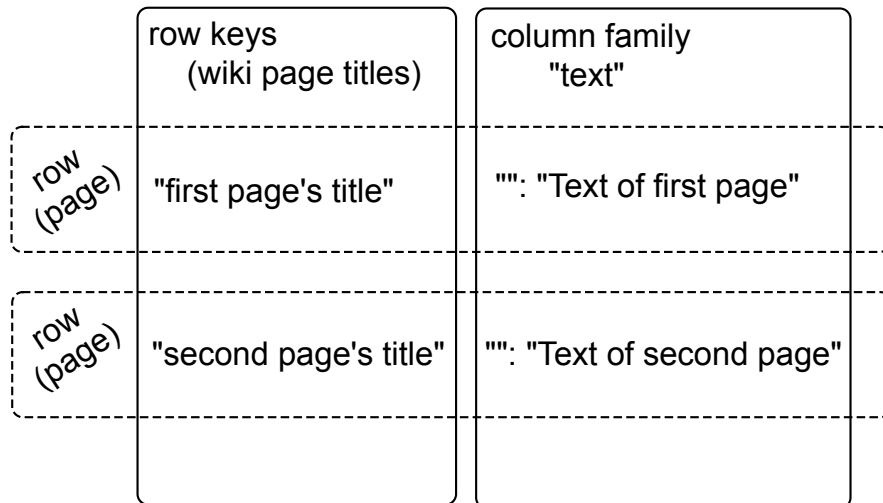
Now let's take what you've learned about table structure and use it to do something fun—you're going to make a wiki! There are lots of juicy info bits you might want to associate with a wiki, but you'll start with the bare minimum. A wiki contains pages, each of which has a unique title string and contains some article text.

Use the create command to make our wiki table in the HBase shell:

```
hbase> create 'wiki', 'text'
0 row(s) in 1.2160 seconds
```

Here, we're creating a table called `wiki` with a single column family called `text`. The table is currently empty; it has no rows and thus no columns. Unlike a relational database, in HBase a column is specific to the row that contains it. Columns don't have to be predefined in something like a `CREATE TABLE` declaration in SQL. For our purposes here, though, we'll stick to a schema, even though it isn't predefined. When we start adding rows, we'll add columns to store data at the same time.

Visualizing our table architecture, we arrive at something like the following figure.



By our own convention, we expect each row to have exactly one column within the text family, qualified by the empty string (`''`). So, the full column name containing the text of a page will be `'text:'`.

For our wiki table to be useful, it's of course going to need content, so let's add some!

Inserting, Updating, and Retrieving Data

Our wiki needs a Home page, so we'll start with that. To add data to an HBase table, use the put command:

```
hbase> put 'wiki', 'Home', 'text:', 'Welcome to the wiki!'
```

This command inserts a new row into the wiki table with the key 'Home', adding 'Welcome to the wiki!' to the column called 'text:'. Note the colon at the end of the column name. This is actually a requirement in HBase if you don't specify a column family in addition to a column (in this case, you're specifying *no column family*).

We can query the data for the 'Home' row using get, which requires two parameters: the table name and the row key. You can optionally specify a list of columns to return. Here, we'll fetch the value of the text: column:

```
hbase> get 'wiki', 'Home', 'text:'
COLUMN      CELL
text:       timestamp=1295774833226, value=Welcome to the wiki!
1 row(s) in 0.0590 seconds
```

Notice the timestamp field in the output. HBase stores an integer timestamp for all data values, representing time in milliseconds since the epoch (00:00:00 UTC on January 1, 1970). When a new value is written to the same cell, the old value hangs around, indexed by its timestamp. This is a pretty awesome feature, and one that is unique to HBase amongst the databases in this book. Most databases require you to specifically handle historical data yourself, but in HBase, versioning is baked right in!

Finally, let's perform a scan operation:

```
hbase> scan 'wiki'
```

Scan operations simply return all rows in the entire table. Scans are powerful and great for development purposes but they are also a very blunt instrument, so use them with care. We don't have much data in our wiki table so it's perfectly fine, but if you're running HBase in production, stick to more precise reads or you'll put a lot of undue strain on your tables.

Putting and Getting

The put and get commands allow you to specify a timestamp explicitly. If using milliseconds since the epoch doesn't strike your fancy, you can specify another integer value of your choice. This gives you an extra dimension to work with if you need it. If you don't specify a timestamp, HBase will use the current time when inserting, and it will return the most recent version when reading.



Luc says:

Rows Are Like Mini Databases

Rows in HBase are a bit tough to fully understand at first because rows tend to be much more “shallow” in other databases. In relational databases, for example, rows contain any number of column values but not metadata such as timestamps, and they don’t contain the kind of depth that HBase rows do (like the Python dictionary in the previous example).

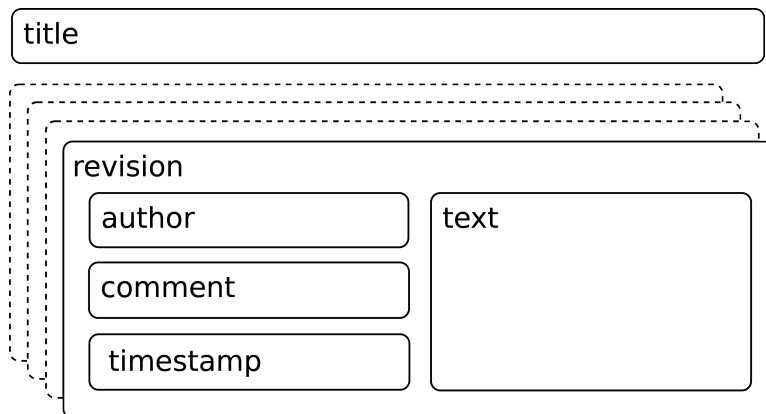
I recommend thinking of HBase rows as being a tiny database in their own right. Each cell in the database can have many different values associated with it (like a mini timeseries database). When you fetch a row in HBase, you’re not fetching a set of values; you’re fetching a small world.

Altering Tables

So far, our wiki schema has pages with titles, text, and an integrated version history but nothing else. Let’s expand our requirements to include the following:

- In our wiki, a page is uniquely identified by its title.
- A page can have unlimited revisions.
- A revision is identified by its timestamp.
- A revision contains text and optionally a commit comment.
- A revision was made by an author, identified by name.

Visually, our requirements can be sketched as you see in the following figure.



In this abstract representation of our requirements for a page, we see that each revision has an author, a commit comment, some article text, and a timestamp. The title of a page is not part of a revision because it’s the identifier we use to

denote revisions belonging to the same page and thus cannot change. If you *did* want to change the title of a page, you'd need to write a whole new row.

Mapping our vision to an HBase table takes a somewhat different form, as illustrated in the figure that follows.

	keys (title)	family "text"	family "revision"
row (page)	"first page"	"": "..."	"author": "..." "comment": "..."
row (page)	"second page"	"": "..."	"author": "..." "comment": "..."

Our wiki table uses the title as the row key and will group other page data into two column families called text and revision. The text column family is the same as before; we expect each row to have exactly one column, qualified by the empty string (""), to hold the article contents. The job of the revision column family is to hold other revision-specific data, such as the author and commit comment.

Defaults

We created the wiki table with no special options, so all the HBase default values were used. One such default value is to keep only three VERSIONS of column values, so let's increase that. To make schema changes, first we have to take the table offline with the disable command.

```
hbase> disable 'wiki'
0 row(s) in 1.0930 seconds
```

Now we can modify column family characteristics using the alter command.

```
hbase> alter 'wiki', { NAME => 'text', VERSIONS =>
hbase*   org.apache.hadoop.hbase.HConstants::ALL_VERSIONS }
0 row(s) in 0.0430 seconds
```

Here, we're instructing HBase to alter the text column family's VERSIONS attribute. There are a number of other attributes we could have set, some of

which we'll discuss in Day 2. The `hbase*` line means that it's a continuation of the previous line.

Altering a Table

Operations that alter column family characteristics can be very expensive because HBase has to create a new column family with the chosen specifications and then copy all the data over. In a production system, this may incur significant downtime. For this reason, the sooner you settle on column family options the better.

With the wiki table still disabled, let's add the revision column family, again using the alter command:

```
hbase> alter 'wiki', { NAME => 'revision', VERSIONS =>
hbase*   org.apache.hadoop.hbase.HConstants::ALL_VERSIONS }
0 row(s) in 0.0660 seconds
```

Just as before, with the text family, we're only adding a revision *column family* to the table schema, not individual *columns*. Though we expect each row to eventually contain a revision:author and revision:comment, it's up to the client to honor this expectation; it's not written into any formal schema. If someone wants to add a revision:foo for a page, HBase won't stop them.

Moving On

With these additions in place, let's reenable our wiki:

```
hbase> enable 'wiki'
0 row(s) in 0.0550 seconds
```

Now that our wiki table has been modified to support our growing requirements list, we can start adding data to columns in the revision column family.

Adding Data Programmatically

As you've seen, the HBase shell is great for tasks such as manipulating tables. Sadly, the shell's data insertion support isn't the best. The `put` command allows you to set only one column value at a time, and in our newly updated schema, we need to add multiple column values simultaneously so they all share the same timestamp. We're going to need to start scripting.

The following script can be executed directly in the HBase shell because the shell is also a JRuby interpreter. When run, it adds a new version of the text for the Home page, setting the author and comment fields at the same time. JRuby runs on the Java virtual machine (JVM), giving it access to the HBase Java code. These examples will *not* work with non-JVM Ruby.

```
hbase/put_multiple_columns.rb
import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'

def jbytes(*args)
  args.map {|arg| arg.to_s.to_java_bytes}
end

table = HTable.new(@hbase.configuration, "wiki")

p = Put.new(*jbytes("Home"))

p.add(*jbytes("text", "", "Hello world"))
p.add(*jbytes("revision", "author", "jimbo"))
p.add(*jbytes("revision", "comment", "my first edit"))

table.put(p)
```

The import lines bring references to useful HBase classes into the shell. This saves us from having to write out the full namespace later. Next, the `jbytes()` function takes any number of arguments and returns an array converted to Java byte arrays, as the HBase API methods demand.

After that, we create a local variable (`table`) pointing to our wiki table, using the `@hbase` administration object for configuration information.

Next, we stage a commit operation by creating and preparing a new instance of a `Put` object, which takes the row to be modified. In this case, we're sticking with the `Home` page we've been working with thus far. Finally, we `add()` properties to our `Put` instance and then call on the `table` object to execute the `put` operation we've prepared. The `add()` method has several forms; in our case, we used the three-argument version: `add(column_family, column_qualifier, value)`.

Why Column Families?

You may be tempted to build your whole structure without column families. Why not just store all of a row's data in a single column family? That solution would be simpler to implement. But there are downsides to avoiding column families. One of them is that you'd miss out on fine-grained performance tuning. Each column family's performance options are configured independently. These settings affect things such as read and write speed and disk space consumption.

The other advantage to keep in mind is that column families are stored in different directories. When reading row data in HBase, you can potentially target your reads to specific column families within the row and thus avoid unnecessary cross-directory lookups, which can provide a speed boost, especially in read-heavy workloads.

All operations in HBase are atomic at the *row level*. No matter how many columns are affected, the operation will have a consistent view of the particular row being accessed or modified. This design decision helps clients reason intelligently about the data.

Our put operation affects several columns and doesn't specify a timestamp, so all column values will have the same timestamp (the current time in milliseconds). Let's verify by invoking get.

```
hbase> get 'wiki', 'Home'
COLUMN                                CELL
revision:author                       timestamp=1296462042029, value=jimbo
revision:comment                      timestamp=1296462042029, value=my first edit
text:                                timestamp=1296462042029, value=Hello world
3 row(s) in 0.0300 seconds
```

As you can see, each column value listed previously has the same timestamp.

Day 1 Wrap-Up

Today, you got a firsthand look at a running HBase server. You learned how to configure it and monitor log files for troubleshooting. And using the HBase shell you performed basic administration and data manipulation tasks.

In providing a basic data model for a wiki storage engine, you explored schema design in HBase. You learned how to create tables and manipulate column families. Designing an HBase schema means making choices about column family options and, just as important, our semantic interpretation of features such as timestamps and row keys.

You also started poking around in the HBase Java API by executing JRuby code in the shell. In Day 2, you'll take this a step further, using the shell to run custom scripts for big jobs such as data import.

At this point, we hope you've been able to uncouple your thinking from relational database terms such as *table*, *row*, and *column*. By all means, don't *forget* those terms; just suspend their meaning in your head for a while longer, as the difference between how HBase uses these terms and what they mean in other systems will become even starker as we delve deeper into HBase's features.

Day 1 Homework

HBase documentation online generally comes in two flavors: extremely technical and nonexistent. There are some decent "getting started" guides out there, but there's a chance you may need to spend some time trawling through Javadoc or source code to find answers.

Find

1. Figure out how to use the shell to do the following:
 - Delete individual column values in a row
 - Delete an entire row
2. Bookmark the HBase API documentation for the version of HBase you're using.

Do

1. Create a function called `put_many()` that creates a `Put` instance, adds any number of column-value pairs to it, and commits it to a table. The signature should look like this:

```
def put_many(table_name, row, column_values)
  # your code here
end
```

2. Define your `put_many()` function by pasting it in the HBase shell, and then call it like so:

```
hbase> put_many 'wiki', 'Some title', {
hbase*   "text:" => "Some article text",
hbase*   "revision:author" => "jschmoe",
hbase*   "revision:comment" => "no comment" }
```

Day 2: Working with Big Data

With Day 1's table creation and manipulation under our belts, it's time to start adding some serious data to our wiki table. Today, you'll script against the HBase APIs, ultimately streaming Wikipedia content right into our wiki! Along the way, you'll pick up some performance tricks for making faster import jobs. Finally, you'll poke around in HBase's internals to see how it partitions data into regions, achieving a series of both performance and disaster recovery goals.

Importing Data, Invoking Scripts

One common problem people face when trying a new database system is how to migrate data into it. Handcrafting `Put` operations with static strings, as you did in Day 1, is all well and good, but you can do better.

Fortunately, pasting commands into the shell is not the only way to execute them. When you start the HBase shell from the command line, you can

specify the name of a JRuby script to run. HBase will execute that script as though it were entered directly into the shell. The syntax looks like this:

```
$ ${HBASE_HOME}/bin/hbase shell <your_script> [<optional_arguments> ...]
```

Because we're interested specifically in “Big Data,” let's create a script for importing Wikipedia articles into our wiki table. The Wikimedia Foundation, which oversees Wikipedia, Wictionary, and other projects, periodically publishes data dumps we can use. These dumps are in the form of enormous XML files. Here's an example record from the English Wikipedia:

```
<page>
  <title>Anarchism</title>
  <id>12</id>
  <revision>
    <id>408067712</id>
    <timestamp>2011-01-15T19:28:25Z</timestamp>
    <contributor>
      <username>RepublicanJacobite</username>
      <id>5223685</id>
    </contributor>
    <comment>Undid revision 408057615 by [[Special:Contributions...</comment>
    <text xml:space="preserve">{{Redirect|Anarchist|the fictional character|
...
[[bat-smg:Anarkēzmos]]
  </text>
</revision>
</page>
```

Because we have such incredible foresight, the individual items in these XML files contain all the information we've already accounted for in our schema: title (row key), text, timestamp, and author. We ought to be able to write a script to import revisions without too much trouble.

Streaming XML

First things first: We'll need to parse the huge XML files in a streaming fashion, so let's start with that. The basic outline for parsing an XML file in JRuby, record by record, looks like this:

```
hbase/basic_xml_parsing.rb
import 'javax.xml.stream.XMLStreamConstants'

factory = javax.xml.stream.XMLInputFactory.newInstance
reader = factory.createXMLStreamReader(java.lang.System.in)

while reader.has_next
  type = reader.next
```

```

if type == XMLStreamConstants::START_ELEMENT
  tag = reader.local_name
  # do something with tag
elsif type == XMLStreamConstants::CHARACTERS
  text = reader.text
  # do something with text
elsif type == XMLStreamConstants::END_ELEMENT
  # same as START_ELEMENT
end
end
end

```

Breaking this down, there are a few parts worth mentioning. First, we produce an `XMLStreamReader` and wire it up to `java.lang.System.in`, which means it will be reading from standard input.

Next, we set up a while loop, which will continuously pull out tokens from the XML stream until there are none left. Inside the while loop, we process the current token. What happens then depends on whether the token is the start of an XML tag, the end of a tag, or the text in between.

Streaming Wikipedia

Now we can combine this basic XML processing framework with our previous exploration of the `HTable` and `Put` interfaces you explored previously. Here is the resultant script. Most of it should look familiar, and we will discuss a few novel parts.

```

hbase/import_from_wikipedia.rb
require 'time'

import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'
import 'javax.xml.stream.XMLStreamConstants'

def jbytes(*args)
  args.map { |arg| arg.to_s.to_java_bytes }
end

factory = javax.xml.stream.XMLInputFactory.newInstance
reader = factory.createXMLStreamReader(java.lang.System.in)

document = nil
buffer = nil
count = 0

table = HTable.new(@hbase.configuration, 'wiki')
table.setAutoFlush(false)

while reader.has_next
  type = reader.next

```

```

if type == XMLStreamConstants::START_ELEMENT
  case reader.local_name
  when 'page' then document = {}
  when /title|timestamp|username|comment|text/ then buffer = []
  end

elsif type == XMLStreamConstants::CHARACTERS
  buffer << reader.text unless buffer.nil?

elsif type == XMLStreamConstants::END_ELEMENT
  case reader.local_name
  when /title|timestamp|username|comment|text/
    document[reader.local_name] = buffer.join
  when 'revision'
    key = document['title'].to_java_bytes
    ts = (Time.parse document['timestamp']).to_i

    p = Put.new(key, ts)
    p.add(*jbytes("text", "", document['text']))
    p.add(*jbytes("revision", "author", document['username']))
    p.add(*jbytes("revision", "comment", document['comment']))
    table.put(p)

    count += 1
    table.flushCommits() if count % 10 == 0
    if count % 500 == 0
      puts "#{count} records inserted (#{document['title']})"
    end
  end
end
end
end
table.flushCommits()
exit

```

A few things to note in the preceding snippet:

- Several new variables were introduced:
 - document holds the current article and revision data.
 - buffer holds character data for the current field within the document (text, title, author, and so on).
 - count keeps track of how many articles you've imported so far.
- Pay special attention to the use of `table.setAutoFlush(false)`. In HBase, data is *automatically flushed* to disk periodically. This is preferred in most applications. By disabling autoflush in our script, any put operations you execute will be buffered until you call `table.flushCommits()`. This allows you to batch writes together and execute them when it's convenient for you.

- If the start tag is a <page>, then reset document to an empty hash. Otherwise, if it's another tag you care about, reset buffer for storing its text.
- We handle character data by appending it to the buffer.
- For most closing tags, you just stash the buffered contents into the document. If the closing tag is a </revision>, however, you create a new Put instance, fill it with the document's fields, and submit it to the table. After that, you use flushCommits() if you haven't done so in a while and report progress to stdout.

Compression and Bloom Filters

We're almost ready to run the script; we just have one more bit of housecleaning to do first. The text column family is going to contain big blobs of text content. Reading those values will take much longer than values like Hello world or Welcome to the wiki! from Day 1. HBase enables us to compress that data to speed up reads:

```
hbase> alter 'wiki', {NAME=>'text', COMPRESSION=>'GZ', BLOOMFILTER=>'ROW'}
0 row(s) in 0.0510 seconds
```

HBase supports two compression algorithms: Gzip (GZ) and Lempel-Ziv-Oberhumer (LZO). The HBase community highly recommends using LZO over Gzip pretty much unilaterally, but here we're using Gzip. Why is that?

The problem with LZO for our purposes here is the implementation's license. While open source, LZO is not compatible with Apache's licensing philosophy, so LZO can't be bundled with HBase. Detailed instructions are available online for installing and configuring LZO support. If you want high-performance compression, use LZO in your own projects.

A *Bloom filter* is a really cool data structure that efficiently answers the question “Have I ever seen this thing before?” and is used to prevent expensive queries that are doomed to fail (that is, to return no results). Originally developed by Burton Howard Bloom in 1970 for use in spell-checking applications, Bloom filters have become popular in data storage applications for determining quickly whether a key exists.

HBase supports using Bloom filters to determine whether a particular column exists for a given row key (BLOOMFILTER=>'ROWCOL') or just whether a given row key exists at all (BLOOMFILTER=>'ROW'). The number of columns within a column family and the number of rows are both potentially unbounded. Bloom filters offer a fast way of determining whether data exists before incurring an expensive disk read.

How Do Bloom Filters Work?

Without going too deep into implementation details, a Bloom filter manages a statically sized array of bits initially set to 0. Each time a new blob of data is presented to the filter, some of the bits are flipped to 1. Determining which bits to flip depends on generating a hash from the data and turning that hash into a set of bit positions.

Later, to test whether the filter has been presented with a particular blob in the past, the filter figures out which bits would have to be 1 and checks them. If any are 0, then the filter can unequivocally say “no.” If all of the bits are 1, then it reports “yes.” Chances are it has been presented with that blob before, but false positives are increasingly likely as more blobs are entered.

This is the trade-off of using a Bloom filter as opposed to a simple hash. A hash will never produce a false positive, but the space needed to store that data is unbounded. Bloom filters use a constant amount of space but will occasionally produce false positives at a predictable rate based on saturation. False positives aren’t a huge deal, though; they just mean that the filter *says* a value is likely to be there, but you will eventually find out that it isn’t.

Engage!

Now that we’ve dissected the script a bit and added some powerful capabilities to our table, we’re ready to kick off the script. Remember that these files are enormous, so downloading and unzipping them is pretty much out of the question. So, what are we going to do?

Fortunately, through the magic of *nix pipes, we can download, extract, and feed the XML into the script all at once. The command looks like this:

```
$ curl https://url-for-the-data-dump.com | bzcata | \
${HBASE_HOME}/bin/hbase shell import_from_wikipedia.rb
```

Note that you should replace the preceding dummy URL with the URL of a WikiMedia Foundation dump file of some kind.² You should use [project]-latest-pages-articles.xml.bz2 for either the English Wikipedia (~12.7 GB)³ or the English Wiktionary (~566 MB).⁴ These files contain all of the most recent revisions of pages in the Main namespace. That is, they omit user pages, discussion pages, and so on.

Plug in the URL and run it! You should start seeing output like this shortly:

2. <https://dumps.wikimedia.org/enwiki/latest>
3. <https://dumps.wikimedia.org/enwiki/latest/enwiki-latest-pages-articles.xml.bz2>
4. <https://dumps.wikimedia.org/enwiktionary/latest/enwiktionary-latest-pages-articles.xml.bz2>

```
500 records inserted (Ashmore and Cartier Islands)
1000 records inserted (Annealing)
1500 records inserted (Ajanta Caves)
```

The script will happily chug along as long as you let it or until it encounters an error, but you'll probably want to shut it off after a while. When you're ready to kill the script, press **Ctrl+C**. For now, though, let's leave it running so we can take a peek under the hood and learn about how HBase achieves its horizontal scalability.

Introduction to Regions and Monitoring Disk Usage

In HBase, rows are kept in order, sorted by the row key. A region is a chunk of rows, identified by the starting key (inclusive) and ending key (exclusive). Regions never overlap, and each is assigned to a specific region server in the cluster. In our simplistic standalone server, there is only one region server, which will always be responsible for all regions. A fully distributed cluster would consist of many region servers.

So, let's take a look at your HBase server's disk usage, which will give us insight into how the data is laid out. You can inspect HBase's disk usage by opening a command prompt to the data/default directory in the `hbase.rootdir` location you specified earlier and executing the `du` command. `du` is a standard *nix command-line utility that tells you how much space is used by a directory and its children, recursively. The `-h` option tells `du` to report numbers in human-readable form.

Here's what ours looked like after about 68 MB worth of pages (out of over 560 MB total or about 160,000 pages) had been inserted and the import was still running:

```
$ du -h
4.0K  ./wiki/.tabledesc
0B    ./wiki/.tmp
0B    ./wiki/1e157605a0e5a1493e4cc91d7e368b05/.tmp
0B    ./wiki/1e157605a0e5a1493e4cc91d7e368b05/recovered.edits
11M   ./wiki/1e157605a0e5a1493e4cc91d7e368b05/revision
64M   ./wiki/1e157605a0e5a1493e4cc91d7e368b05/text
75M   ./wiki/1e157605a0e5a1493e4cc91d7e368b05
75M   ./wiki
75M   .
```

This output tells us a lot about how much space HBase is using and how it's allocated. The lines starting with `/wiki` describe the space usage for the wiki table. The long-named subdirectory `1e157605a0e5a1493e4cc91d7e368b05` represents an individual region (the only region so far). Under that, the directories `/text`

and `/revision` correspond to the text and revision column families, respectively. Finally, the last line sums up all these values, telling us that HBase is using 75 MB of disk space. We can safely ignore the `.tmp` and `recovered.edits` for now.

One more thing. In the directory that you specified using the `hbase.rootdir` variable, you'll find three folders named `MasterProcWALs`, `WALs`, and `oldWALs`. These folders hold write-ahead log (WAL) files. HBase uses write-ahead logging to provide protection against node failures. This is a fairly typical disaster recovery technique. For instance, write-ahead logging in file systems is called *journaling*. In HBase, logs are appended to the WAL before any edit operations (put and increment) are persisted to disk.

For performance reasons, edits are not necessarily written to disk immediately. The system does much better when I/O is buffered and written to disk in chunks. If the *region server* responsible for the affected region were to crash during this limbo period, HBase would use the WAL to determine which operations were successful and take corrective action. Without a WAL, a region server crash would mean that that not-yet-written data would be simply lost.

Writing to the WAL is optional and enabled by default. Edit classes such as `Put` and `Increment` have a setter method called `setWriteToWAL()` that can be used to exclude the operation from being written to the WAL. Generally you'll want to keep the default option, but in some instances it might make sense to change it. For example, if you're running an import job that you can rerun any time, such as our Wikipedia import script, you might prioritize the performance benefit of disabling WAL writes over disaster recovery protection.

Regional Interrogation

If you let the script run long enough, you'll see HBase split the table into multiple regions. Here's our `du` output again, after about 280 MB worth of data (roughly 2.1 million pages) has been written to the `wiki` table:

```
$ du -h
4.0K    ./wiki/.tabledesc
0B      ./wiki/.tmp
0B      ./wiki/48576fdcfc9b29257fb93d33dbeda90/.tmp
0B      ./wiki/48576fdcfc9b29257fb93d33dbeda90/recovered.edits
132M    ./wiki/48576fdcfc9b29257fb93d33dbeda90/revision
145M    ./wiki/48576fdcfc9b29257fb93d33dbeda90/text
277M    ./wiki/48576fdcfc9b29257fb93d33dbeda90
0B      ./wiki/bd36620826a14025a35f1fe5e928c6c9/.tmp
0B      ./wiki/bd36620826a14025a35f1fe5e928c6c9/recovered.edits
134M    ./wiki/bd36620826a14025a35f1fe5e928c6c9/revision
113M    ./wiki/bd36620826a14025a35f1fe5e928c6c9/text
```

```
247M    ./wiki/bd36620826a14025a35f1fe5e928c6c9
1.0G    ./wiki
1.0G    .
```

The biggest change is that the old region (1e157605a0e5a1493e4cc91d7e368b05) is now gone and has been replaced by two new regions (48576fd... and bd36620.... In our stand-alone server, all the regions are served by our single server, but in a distributed environment these would be parceled across multiple *region* servers.

This raises a few questions, such as “How do the region servers know which regions they’re responsible for serving?” and “How can you find which region (and, by extension, which region server) is serving a given row?”

If we drop back into the HBase shell, we can query the `hbase:meta` to find out more about the current regions. `hbase:meta` is a special table whose sole purpose is to keep track of all the user tables and which region servers are responsible for serving the regions of those tables.

```
hbase> scan 'hbase:meta', { COLUMNS => ['info:server', 'info:regioninfo'] }
```

Even for a small number of regions, you should get a lot of output. Let’s just focus on the rows that begin with `wiki` for now. Here’s a fragment of ours, formatted and truncated for readability:

```
ROW
wiki,,1487399747176.48576fdcfcb9b29257fb93d33dbeda90.

COLUMN+CELL
column=info:server, timestamp=..., value=localhost.localdomain:35552
column=info:regioninfo, timestamp=1487399747533, value={
  ENCODED => 48576fdcfcb9b29257fb93d33dbeda90,
  NAME => 'wiki,,1487399747176.48576fdcfcb9b29257fb93d33dbeda90.',
  STARTKEY => '', ENDKEY => 'lacrimamj'}

ROW
wiki,lacrimamj,1487399747176.bd36620826a14025a35f1fe5e928c6c9.

COLUMN+CELL
column=info:server, timestamp=..., value=localhost.localdomain:35552
column=info:regioninfo, timestamp=1487399747533, value={
  ENCODED => bd36620826a14025a35f1fe5e928c6c9,
  NAME => 'wiki,lacrimamj,1487399747176.bd36620826a14025a35f1fe5e928c6c9.',
  STARTKEY => 'lacrimamj',
  ENDKEY => ''}
```

Both of the regions listed previously are served by the same server, `localhost.localdomain:35552`. The first region starts at the empty string row (`"`) and ends with `'lacrimamj'`. The second region starts at `'lacrimamj'` and goes to `"` (that is, to the end of the available keyspace).

STARTKEY is inclusive, while ENDKEY is exclusive. So, if you were looking for the 'Demographics of Macedonia' row, you'd find it in the first region.

Because rows are kept in sorted order, we can use the information stored in hbase:meta to look up the region and server where any given row should be found. But where is the hbase:meta table stored?

It turns out that the hbase:meta table can also be split into regions and served by region servers just like any other table would be. If you run the load script as long as we have, this may or may not happen on your machine; you may have this table stored in only one region. To find out which servers have which parts of the hbase:meta table, look at the results of the preceding scan query but pay attention to the rows that begin with hbase:namespace.

```
ROW
hbase:namespace,,1486625601612.aa5b4cfb7204bfc50824dee1886103c5.
COLUMN+CELL
column=info:server, timestamp=..., value=localhost.localdomain:35552
column=info:regioninfo, timestamp=1486625602069, value={
  ENCODED => aa5b4cfb7204bfc50824dee1886103c5,
  NAME => 'hbase:namespace,,1486625601612.aa5b4cfb7204bfc50824dee1886103c5.',
  STARTKEY => '',
  ENDKEY => ''}
```

In this case, the entire keyspace (beginning with " and ending with ") is stored in the aa5b4cfb7204bfc50824dee1886103c5, which is on disk on our machine in the data/hbase/namespace/aa5b4cfb7204bfc50824dee1886103c5 subdirectory of our HBase data folder (your region name will vary).

Describe Your Tables

To see other metadata associated with an HBase table, use the describe command, like so:

```
hbase> describe 'wiki'
hbase> describe 'hbase:meta'
```

This will tell you whether the table is currently enabled and provide a lot of information about each column family in the table, including any Bloom filters that you've applied, which compression is used, and so on.

The assignment of regions to region servers, including hbase:meta regions, is handled by the *master* node, often referred to as HBaseMaster. The master server can also be a region server, performing both duties simultaneously.

When a region server fails, the master server steps in and reassigns responsibility for regions previously assigned to the failed node. The new stewards

of those regions would look to the WAL to see what, if any, recovery steps are needed. If the master server fails, responsibility defers to any of the other region servers that step up to become the master.

Scanning One Table to Build Another

Once you've stopped the import script from running, we can move on to the next task: extracting information from the imported wiki contents.

Wiki syntax is filled with links, some of which link internally to other articles and some of which link to external resources. This interlinking contains a wealth of topological data. Let's capture it!

Our goal is to capture the relationships between articles as directional links, pointing one article *to* another or receiving a link *from* another. An internal article link in wikitext looks like this: `[[<target name>|<alt text>]]`, where `<target name>` is the article to link to, and `<alt text>` is the alternative text to display (optional).

For example, if the text of the article on *Star Wars* contains the string `"[[Yoda|jedi master]]"`, we want to store that relationship twice—once as an outgoing link from *Star Wars* and once as an incoming link to Yoda. Storing the relationship twice means that it's fast to look up both a page's outgoing links and its incoming links.

To store this additional link data, we'll create a new table. Head over to the shell and enter this:

```
hbase> create 'links', {
  NAME => 'to', VERSIONS => 1, BLOOMFILTER => 'ROWCOL'
},{
  NAME => 'from', VERSIONS => 1, BLOOMFILTER => 'ROWCOL'
}
```

In principle, we could have chosen to shove the link data into an existing column family or merely added one or more additional column families to the wiki table rather than create a new one. When you create a separate table, this has the advantage that the tables have separate regions, which in turn means that the cluster can more effectively split regions as necessary.

The general guidance for column families in the HBase community is to try to keep the number of families per table down. You can do this either by combining more columns into the same families or by putting families in different tables entirely. The choice is largely decided by whether and how often clients will need to get an entire row of data (as opposed to needing just a few column values).

For the wiki application we've been developing, the text and revision column families need to be on the same table so when you put new revisions in, the metadata and the text share the same timestamp. The links content, by contrast, will never have the same timestamp as the article from which the data came. Further, most client actions will be interested either in the article text or in the extracted information about article links but probably not in both at the same time. So, splitting out the to and from column families into a separate table makes sense.

Constructing the Scanner

With the links table created, we're ready to implement a script that'll scan all the rows of the wiki table. Then, for each row, it'll retrieve the wikitext and parse out the links. Finally, for each link found, it'll create incoming and outgoing link table records. The bulk of this script should be pretty familiar to you by now. Most of the pieces are recycled, and we'll discuss the few novel bits.

`hbase/generate_wiki_links.rb`

```
import 'org.apache.hadoop.hbase.client.HTable'
import 'org.apache.hadoop.hbase.client.Put'
import 'org.apache.hadoop.hbase.client.Scan'
import 'org.apache.hadoop.hbase.util.Bytes'

def jbytes(*args)
  return args.map { |arg| arg.to_s.to_java_bytes }
end

wiki_table = HTable.new(@hbase.configuration, 'wiki')
links_table = HTable.new(@hbase.configuration, 'links')
links_table.setAutoFlush(false)

scanner = wiki_table.getScanner(Scan.new)

linkpattern = /[^\[\(\^\[\]\|\\:\#\][^\[\]\|:]*)?(:\[^\[\]\|]+\])?\[\]\]/
count = 0

while (result = scanner.next())
  title = Bytes.toString(result.getRow())
  text = Bytes.toString(result.getValue(*jbytes('text', '')))
  if text
    put_to = nil
    text.scan(linkpattern) do |target, label|
      unless put_to
        put_to = Put.new(*jbytes(title))
        put_to.setWriteToWAL( false )
      end

      target.strip!
      target.capitalize!

      label = '' unless label
      label.strip!
```

```

    put_to.add(*jbytes("to", target, label))
    put_from = Put.new(*jbytes(target))
    put_from.add(*jbytes("from", title, label))
    put_from.setWriteToWAL(false)
    links_table.put(put_from)
  end
  links_table.put(put_to) if put_to
  links_table.flushCommits()
end

count += 1
puts "#{count} pages processed (#{title})" if count % 500 == 0
end

links_table.flushCommits()
exit

```

A few things to note in this script:

- First, we grab a Scan object, which we'll use to scan through the wiki table.
- Extracting row and column data requires some byte wrangling but generally isn't too bad either.
- Each time the linkpattern appears in the page text, we extract the target article and text of the link and then use those values to add to our Put instances.
- Finally, we tell the table to execute our accumulated Put operations. It's possible (though unlikely) for an article to contain no links at all, which is the reason for the if put_to clause.
- Using setWriteToWAL(false) for these puts is a judgment call. Because this exercise is for educational purposes and because you could simply rerun the script if anything went wrong, we'll take the speed bonus and accept our fate should the node fail.

Running the Script

If you're ready to throw caution to the wind, run the script.

```
${HBASE_HOME}/bin/hbase shell generate_wiki_links.rb
```

It should produce output like this:

```

500 pages processed (10 petametres)
1000 pages processed (1259)
1500 pages processed (1471 BC)
2000 pages processed (1683)
...

```

As with the previous script, you can let it run as long as you like, even to completion. If you want to stop it, press **Ctrl+C**.

You can monitor the disk usage of the script using `du` as we've done before. You'll see new entries for the links table we just created, and the size counts will increase as the script runs.

Examining the Output

We just created a scanner programmatically to perform a sophisticated task. Now we'll use the shell's `scan` command to simply dump part of a table's contents to the console. For each link the script finds in a text: blob, it will indiscriminately create both to and from entries in the links table. To see the kinds of links being created, head over to the shell and scan the table.

```
hbase> scan 'links', STARTROW => "Admiral Ackbar", ENDROW => "It's a Trap!"
```

You should get a whole bunch of output. Of course, you can use the `get` command to see the links for just a single article.

```
hbase> get 'links', 'Addition'
```

COLUMN	CELL
from:+	timestamp=1487402389072, value=
from:0	timestamp=1487402390828, value=
from:Addition	timestamp=1487402391595, value=
from:Appendix:Basic English word list	timestamp=1487402393334, value=
...	

The structure of the wiki table is highly regular, with each row consisting of the same columns. As you recall, each row has `text:`, `revision:`, `author:`, and `revision:` `comment` columns. The links table has no such regularity. Each row may have one column or hundreds. And the variety of column names is as diverse as the row keys themselves (titles of Wikipedia articles). That's okay! HBase is a so-called sparse data store for exactly this reason.

To find out just how many rows are now in your table, you can use the `count` command.

```
hbase> count 'wiki', INTERVAL => 100000, CACHE => 10000
```

Current count: 100000, row: Nov-Zelandon
Current count: 200000, row: adjudicamur
Current count: 300000, row: aquatores
Current count: 500000, row: coiso
...
Current count: 1300000, row: occludesti
Current count: 1400000, row: plendonta
Current count: 1500000, row: receptarum
Current count: 1900000, row: ventilators
2179230 row(s) in 17.3440 seconds

Because of its distributed architecture, HBase doesn't immediately know how many rows are in each table. To find out, it has to count them (by performing a table scan). Fortunately, HBase's region-based storage architecture lends itself to fast distributed scanning. So, even if the operation at hand requires a table scan, we don't have to worry quite as much as we would with other databases.

Day 2 Wrap-Up

Whew, that was a pretty big day! You learned how to write an import script for HBase that parses data out of a stream of XML. Then you used those techniques to stream Wikipedia dumps directly into your wiki table.

You learned more of the HBase API, including some client-controllable performance levers such as `setAutoFlush()`, `flushCommits()`, and `setWriteToWAL()`. Along those lines, we discussed some HBase architectural features such as disaster recovery, provided via the write-ahead log.

Speaking of architecture, you discovered table regions and how HBase divvies up responsibility for them among the region servers in the cluster. We scanned the `hbase:meta` table to get a feel for HBase internals.

Finally, we discussed some of the performance implications of HBase's sparse design. In so doing, we touched on some community best practices regarding the use of columns, families, and tables.

Day 2 Homework

Find

1. Find a discussion or article describing the pros and cons of compression in HBase.
2. Find an article explaining how Bloom filters work in general and how they benefit HBase.
3. Aside from the algorithm, what other column family options relate to compression?
4. How does the type of data and expected usage patterns inform column family compression options?

Do

Expanding on the idea of data import, let's build a database containing nutrition facts.

Download the MyPyramid Raw Food Data set from Data.gov⁵ and extract the zipped contents to `Food_Display_Table.xml`.

This data consists of many pairs of `<Food_Display_Row>` tags. Inside these, each row has a `<Food_Code>` (integer value), `<Display_Name>` (string), and other facts about the food in appropriately named tags.

1. Create a new table called `foods` with a single column family to store the facts. What should you use for the row key? What column family options make sense for this data?
2. Create a new JRuby script for importing the food data. Use the streaming XML parsing style we used earlier for the Wikipedia import script and tailor it to the food data. Pipe the food data into your import script on the command line to populate the table.
3. Using the HBase shell, query the `foods` table for information about your favorite foods.

Day 3: Taking It to the Cloud

On Days 1 and 2, you got quite a lot of hands-on experience using HBase in standalone mode. Our experimentation so far has focused on accessing a single local server. But in reality, if you choose to use HBase, you'll want to have a good-sized cluster in order to realize the performance benefits of its distributed architecture. And nowadays, there's also an increasingly high chance that you'll want to run it in the cloud.

Here on Day 3, let's turn our attention toward operating and interacting with a remote HBase cluster. First, you'll deploy an HBase cluster on Amazon Web Services' Elastic MapReduce platform (more commonly known as AWS and EMR, respectively) using AWS's command-line tool, appropriately named `aws`. Then, you'll connect directly to our remote HBase cluster using Secure Shell (SSH) and perform some basic operations.

Initial AWS and EMR Setup

EMR is a managed Hadoop platform for AWS. It enables you to run a wide variety of servers in the Hadoop ecosystem—Hive, Pig, HBase, and many others—on EC2 without having to engage in a lot of the nitty-gritty details usually associated with managing those systems.

5. <https://www.cnpp.usda.gov/Innovations/DataSource/MyFoodapediaData.zip>

Warning! AWS Isn't Free

Whenever you're using AWS—or any other cloud provider—always keep in mind that you're using paid services. The exercise that you're about to go through will *probably* be free for you, but it may end up costing a few units of whatever your local currency happens to be. You're free to leave the cluster running, especially if you want to do the Day 3 homework in the next section, but we recommend terminating it whenever you're done so that you don't rack up unwanted costs. To do so at any time, use the `terminate-clusters` command (more on setting a `CLUSTER_ID` environment variable later in this section):

```
$ aws emr terminate-clusters \
  --cluster-ids ${CLUSTER_ID}
```

You can also set up a usage-based alarm using AWS's CloudWatch service in case you want some extra assurance that you won't end up with any unpleasant billing surprises.^a

- a. http://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/monitor_estimated_charges_with_cloudwatch.html

Before you can get started spinning up an HBase cluster, you'll need to sign up for an AWS account.⁶ Once you've created an account, log into the IAM service in the AWS console⁷ and create a new user by clicking Add User.

During the user creation process, select “Programmatic access” and then click “Attach existing policies directly.” Select the following policies: `IAMFullAccess`, `AmazonEC2FullAccess`, and `AmazonElasticMapReduceFullAccess`. Then, fetch your AWS access key and secret key from a different section of the console.⁸ With that information in hand, install the `aws` tool using `pip` and then run `aws --version` to ensure that the tool installed properly. To configure the client, just run:

```
$ aws configure
```

This will prompt you to enter your access key and secret key and two other pieces of information: a default region name (basically which AWS datacenter you'd like to use) and a default output format. Input `us-east-1` and `json` respectively (though feel free to select a different region if you'd like; the authors happen to be partial to `us-west-1` in Oregon). To make sure that your setup is now in place, run `aws emr list-clusters`, which lists the clusters you've created in EMR. That should return an empty list:

6. <http://aws.amazon.com/>

7. <https://console.aws.amazon.com/iam>

8. <https://console.aws.amazon.com/iam>

```
{
  "Clusters": []
}
```

In AWS, your ability to perform actions is based on which *roles* you possess as a user. We won't delve into service access control here. For our purposes, you just need to create a set of roles that enable you to access EMR and to spin up, manage, and finally access clusters. You can create the necessary roles with one convenient built-in command:

```
$ aws emr create-default-roles
```

Once your HBase cluster is up and running in a little bit, you'll need to be able to access it remotely from your own machine. In AWS, direct access to remote processes is typically done over SSH. You'll need to create a new SSH *key pair*, upload it to AWS, and then specify that key pair by name when you create your cluster. Use these commands to create a key pair in your `~/.ssh` directory and assign it restrictive permissions:

```
$ aws ec2 create-key-pair \
  --key-name HBaseShell \
  --query 'KeyMaterial' \
  --output text > ~/.ssh/hbase-shell-key.pem
$ chmod 400 ~/.ssh/hbase-shell-key.pem
```

Now you have a key pair stored in the `hbase-shell-key.pem` file that you can use later to SSH into your cluster. To ensure that it's been successfully created:

```
$ aws ec2 describe-key-pairs
{
  "KeyPairs": [
    {
      "KeyName": "HBaseShell",
      "KeyFingerprint": "1a:2b:3c:4d:1a:..."
    }
  ]
}
```

Creating the Cluster

Now that that initial configuration detour is out of the way, you can get your hands dirty and create your HBase cluster.

```
$ aws emr create-cluster \
  --name "Seven DBs example cluster" \
  --release-label emr-5.3.1 \
  --ec2-attributes KeyName=HBaseShell \
  --use-default-roles \
  --instance-type m1.large \
  --instance-count 3 \
  --applications Name=HBase
```

That's a pretty intricate shell command! Let's break down some of the non-obvious parts.

- `--release-label` specifies which release of EMR you're working with.
- `--ec2-attributes` specifies which key pair you want to use to create the cluster (which will enable you to have SSH access later).
- `--instance-type` specifies which type of machine you want your cluster to run on.
- `--instance-count` is the number of machines you want in the cluster (by default, 3 instances will mean one master node and two slave nodes).
- `--use-default-roles` means that you're using the default roles you created a minute ago.
- `--applications` determines which Hadoop application you'll install (just HBase for us).

If `create-cluster` is successful, you should get a JSON object back that displays the ID of the cluster. Here's an example ID:

```
{
  "ClusterId": "j-1MFV1QTNSBTD8"
}
```

For convenience, store the cluster ID in your environment so it's easier to use in later shell commands. This is always a good practice when working with AWS on the command line, as almost everything has a randomly generated identifier.

```
$ export CLUSTER_ID=j-1MFV1QTNSBTD8
```

You can verify that the cluster has been created by listing all of the clusters associated with your user account.

```
$ aws emr list-clusters
```

That command should now return a JSON object like this:

```
{
  "Clusters": [
    {
      "Status": {
        "Timeline": {
          "CreationDateTime": 1487455208.825
        },
        "State": "STARTING",
        "StateChangeReason": {}
      },
    },
  ],
}
```

```

        "NormalizedInstanceHours": 0,
        "Id": "j-1MFV1QTNSBTD8",
        "Name": "Seven DBs example cluster"
    }
}
]
}

```

At this point, your cluster has been *created* but it will take a while to actually start, usually several minutes. Run this command, which checks every five seconds for the current status of the cluster (you should see "STARTING" at first):

```

$ while true; do
    aws emr describe-cluster \
        --cluster-id ${CLUSTER_ID} \
        --query Cluster.Status.State
    sleep 5
done

```

Again, this could take a while, so take a coffee break, read some EMR documentation, whatever you feel like. Once the the state of the cluster turns to "WAITING", it should be ready to go. You can now inspect all three machines running in the cluster (one master and two slave nodes):

```

$ aws emr list-instances \
    --cluster-id ${CLUSTER_ID}

```

Each instance has its own configuration object associated with it that tells you each instance's current status (RUNNING, TERMINATED, and so on), DNS name, ID, private IP address, and more.

Enabling Access to the Cluster

You have just one last step before you can access your HBase cluster via SSH. You need to authorize TCP *ingress* into the master node of the cluster. To do that, you need to get an identifier for the security group that it belongs to:

```

$ aws emr describe-cluster \
    --cluster-id ${CLUSTER_ID} \
    --query Cluster.Ec2InstanceAttributes.EmrManagedMasterSecurityGroup

```

That should return something like sg-bd63e1ab. Set the SECURITY_GROUP_ID environment variable to that value. Now, you need to run a command that instructs EC2 (which controls the machines running the cluster) to allow TCP ingress on port 22 (used for SSH) from the IP address of your current machine, which you can set as an environment variable.

```
$ export MY_CIDR=$(dig +short myip.opendns.com @resolver1.opendns.com.)/32
$ aws ec2 authorize-security-group-ingress \
  --group-id ${SECURITY_GROUP_ID} \
  --protocol tcp \
  --port 22 \
  --cidr $MY_CIDR
```

Finally, you can SSH into the cluster with the handy `emr ssh` command and point to your local SSH keys and the correct cluster:

```
$ aws emr ssh \
  --cluster-id ${CLUSTER_ID} \
  --key-pair-file ~/.ssh/hbase-shell-key.pem
```

Once the SSH connection is established, you should see a huge ASCII banner whiz by before you're dropped into a remote shell. Now you can open the HBase shell:

```
$ hbase shell
```

If you then see a shell prompt like `hbase(main):001:0>` pop up in your CLI, you've made it! You're now using your own machine as a portal into an HBase cluster running in a datacenter far away (or maybe close by; pretty cool either way). Run a couple other HBase commands from previous exercises for fun:

```
hbase(main):001:0> version
hbase(main):002:0> status
hbase(main):003:0> create 'messages', 'text'
hbase(main):004:0> put 'messages', 'arrival', 'text:', 'HBase: now on AWS!'
hbase(main):005:0> get 'messages', 'arrival'
```

As we mentioned before, always bear in mind that AWS costs money. The exercise that you went through today most likely cost less than a latté at the corner coffee shop. You're free to leave the cluster running, especially if you want to do the Day 3 homework in the next section. You can shut your cluster down at any time using the `terminate-clusters` command:

```
$ aws emr terminate-clusters \
  --cluster-ids ${CLUSTER_ID}
```

Day 3 Wrap-Up

Today you stepped outside of your own machine and installed an HBase cluster in an AWS datacenter, connected your local machine to the remote cluster, played with some of the HBase shell commands that you learned on Day 1, and learned a bit about interacting with AWS services via the command line. This will come in handy when you work with Amazon's DynamoDB and a variety of other AWS services.

Day 3 Homework

For today's homework, open up the AWS documentation for the Find section. For the Do section, leave your HBase cluster running on EMR with the HBase shell open. Just remember to terminate the cluster when you're done!

Find

1. Use the help interface `aws` for the CLI tool to see which commands are available for the `emr` subcommand. Read through the help material for some of these commands to get a sense of some of the capabilities offered by EMR that we didn't cover in today's cluster building exercise. Pay special attention to scaling-related commands.
2. Go to the EMR documentation at <https://aws.amazon.com/documentation/emr> and read up on how to use Simple Storage Service (S3) as a data store for HBase clusters.

Do

1. In your HBase shell that you're accessing via SSH, run some of the cluster metadata commands we explored on Day 2, such as `scan 'hbase:meta'`. Make note of anything that's fundamentally different from what you saw when running HBase locally in standalone mode.
2. Navigate around the EMR section of your AWS browser console⁹ and find the console specific to your running HBase cluster. Resize your cluster down to just two machines by removing one of the slave nodes (known as core nodes). Then increase the cluster size back to three (with two slave/core nodes).
3. Resizing a cluster in the AWS console is nice, but that's not an automatable approach. The `aws` CLI tool enables you to resize a cluster programmatically. Consult the docs for the `emr modify-instance-groups` command by running `aws emr modify-instance-groups help` to find out how this works. Remove a machine from your cluster using that command.

Wrap-Up

HBase is a juxtaposition of simplicity and complexity. Its data storage model is pretty straightforward, with a lot of flexibility and just a few built-in schema constraints. A major barrier to understanding HBase, though, stems from the

9. <https://console.aws.amazon.com/elasticmapreduce>

fact that many terms are overloaded with baggage from the relational world (such as *table* and *column*). Schema design in HBase typically boils down to deciding on the performance characteristics that you want to apply to your tables and columns, which is pretty far afield from the relational world, where things usually hinge upon table design.

HBase's Strengths

Noteworthy features of HBase include a robust scale-out architecture and built-in versioning and compression capabilities. HBase's built-in versioning capability can be a compelling feature for certain use cases. Keeping the version history of wiki pages is a crucial feature for policing and maintenance, for instance. By choosing HBase, you don't have to take any special steps to implement page history—you get it for free. No other database in this book offers that out of the box.

On the performance front, HBase is meant to scale out. If you have huge amounts of data, measured in many terabytes or more, HBase may be for you. HBase is rack aware, replicating data within and between datacenter racks so that node failures can be handled gracefully and quickly.

The HBase community is pretty awesome. There's almost always somebody on the #hbase IRC channel,¹⁰ on HBase's dedicated Slack channel,¹¹ or on the mailing list¹² ready to help with questions and get you pointed in the right direction.

HBase's Weaknesses

Although HBase is designed to scale out, it doesn't scale down. The HBase community seems to agree that five nodes is the minimum number you'll want to use. Because it's designed to be big, it can also be harder to administer (though platforms like EMR, which you saw in Day 3, do provide some good managed options). Solving small problems isn't what HBase is about, and nonexpert documentation is tough to come by, which steepens the learning curve.

Additionally, HBase is almost never deployed alone. Instead, it is usually used in conjunction with other scale-ready infrastructure piece. These include

10. <irc://irc.freenode.net/#hbase>

11. <https://apache-hbase.slack.com/>

12. <http://hbase.apache.org/mail-lists.html>

Hadoop (an implementation of Google’s MapReduce), the Hadoop distributed file system (HDFS), Zookeeper (a headless service that aids internode coordination), and Apache Spark (a popular cluster computing platform). This ecosystem is both a strength and a weakness; it simultaneously provides an ever-expanding set of powerful tools, but making them all work in conjunction with one another across many machines—sometimes in the thousands—can be quite cumbersome.

One noteworthy characteristic of HBase is that it doesn’t offer any sorting or indexing capabilities aside from the row keys. Rows are kept in sorted order by their row keys, but no such sorting is done on any other field, such as column names and values. So, if you want to find rows by something other than their key, you need to either scan the table or maintain your own index (perhaps in a separate HBase table or in an external system).

Another missing concept is datatypes. All field values in HBase are treated as uninterpreted arrays of bytes. There is no distinction between, say, an integer value, a string, and a date. They’re all bytes to HBase, so it’s up to your application to interpret the bytes, which can be tricky, especially if you’re used to relational access patterns like object-relational mappers (ORMs).

HBase on CAP

With respect to CAP, HBase is decidedly CP (for more information on the CAP theorem, see [Appendix 2, *The CAP Theorem*, on page 315](#)). HBase makes strong consistency guarantees. If a client succeeds in writing a value, other clients will receive the updated value on the next request. Some databases allow you to tweak the CAP equation on a per-operation basis. Not so with HBase. In the face of reasonable amounts of partitioning—for example, a node failing—HBase will remain available, shunting the responsibility off to other nodes in the cluster. However, in the pathological case, where only one node is left alive, HBase has no choice but to refuse requests.

The CAP discussion gets a little more complex when you introduce cluster-to-cluster replication, an advanced feature we didn’t cover in this chapter. A typical multicluster setup could have clusters separated geographically by some distance. In this case, for a given column family, one cluster is the system of record, while the other clusters merely provide access to the replicated data. This system is *eventually consistent* because the replication clusters will serve up the most recent values they’re aware of, which may not be the most recent values in the master cluster.

Parting Thoughts

HBase can be quite a challenge at first. The terminology is often deceptively reassuring, and the installation and configuration are not for the faint of heart. On the plus side, some of the features HBase offers, such as versioning and compression, are quite unique. These aspects can make HBase quite appealing for solving certain problems. And of course, it scales out to many nodes of commodity hardware quite well. All in all, HBase—like a nail gun—is a pretty big tool, so watch your thumbs.